Computer Architecture Lab 3

Jonas Van Nieuwenhuyze, Gil Nimmegeers

1 Introduction

In the previous lab session, a significant challenge arose when attempting to find the maximum value in an array larger than 2048 elements. This obstacle prompted the exploration of alternative strategies, leading to the usage of strides. In this lab, we delve deeper into the concepts of warps and thread divergence, both aspects of GPU computing, to gain insights into their impact on performance and how it can be used in image processing.

2 Warps and Thread divergence

Warps are essentially groups of threads that execute instructions in lockstep on a GPU. Each warp typically contains 32 threads, and these threads execute the same instruction simultaneously, but on different data elements.
When working with image processing kernels, such as inverting an image, organizing threads into warps ensures efficient utilization of GPU resources and maximizes parallelism. For example, when applying the inversion operation to each pixel in an image, threads within a warp can process neighboring pixels concurrently. This organization enables efficient memory access patterns and minimizes memory latency, leading to improved performance.
Additionally, warps help manage thread divergence.Thread divergence refers to a situation in parallel computing where threads within the same thread block or warp, execute different code paths. In this assignment, thread divergence arises when processing the R color component differently from the G and B components. For example, when augmenting the inversion kernel to apply a specific transformation to the R component, threads within a warp may need to follow different execution paths depending on the color component being processed.

3 Results

Figure 1 shows the time (ms) in function of the number of threads. The most efficient amount of threads is always a multiple of 32 for less than 192 threads. Every stream multiprocessor is made out of 6 blocks with 32 threads each (= 192 threads). When allocating more than 192 threads, the other SM will also be used. Therefor, the gaps to the most efficient amount of threads grows.
Arranging the RGB array as RR..RRGG..GGBB..BB does not make the program much faster. This is because the program still has to wait for the slower threads to finish while the faster threads are already finished. This does mean that another program could use those faster threads earlier.

Figure 1: This image shows the timings in correlation to the number of threads (20 threads to 1024 threads)

